Working with JavaScript in Rails

This guide covers the built-in Ajax/JavaScript functionality of Rails (and more); it will enable you to create rich and dynamic Ajax applications with ease!

1 An Introduction to Ajax

In order to understand Ajax, you must first understand what a web browser does normally.

When you type `http://localhost:3000` into your browser's address bar and hit 'Go,' the browser (your 'client') makes a request to the server. It parses the response, then fetches all associated assets, like JavaScript files, stylesheets and images. It then assembles the page. If you click a link, it does the same process: fetch the page, fetch the assets, put it all together, show you the results. This is called the 'request response cycle.'

JavaScript can also make requests to the server, and parse the response. It also has the ability to update information on the page. Combining these two powers, a JavaScript writer can make a web page that can update just parts of itself, without needing to get the full page data from the server. This is a powerful technique that we call Ajax.

Rails ships with CoffeeScript by default, and so the rest of the examples in this guide will be in CoffeeScript. All of these lessons, of course, apply to vanilla JavaScript as well.

As an example, here's some CoffeeScript code that makes an Ajax request using the jQuery library:

```
$.ajax(url: "/test").done (html) ->
  $("#results").append html
```

This code fetches data from "/test", and then appends the result to the `div` with an id of `results`.

Rails provides quite a bit of built-in support for building web pages with this technique. You rarely have to write this code yourself. The rest of this guide will show you how Rails can help you write websites in this way, but it's all built on top of this fairly simple technique.

2 Unobtrusive JavaScript

Rails uses a technique called "Unobtrusive JavaScript" to handle attaching JavaScript to the DOM. This is generally considered to be a best-practice within the frontend community, but you may occasionally read tutorials that demonstrate other ways.

Here's the simplest way to write JavaScript. You may see it referred to as 'inline JavaScript':

```
<a href="#" onclick="this.style.backgroundColor='#990000'">Paint it
red</a>
```

When clicked, the link background will become red. Here's the problem: what happens when we have lots of JavaScript we want to execute on a click?